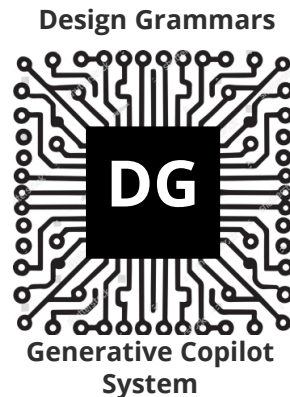


ptBIM 26

DESIGN GRAMMARS SYSTEM SUPPORTING DECISION-MAKING IN CONSTRUCTION ACROSS PLATFORMS AND DISCIPLINES.

Carrying parametric design intent and machine-readable design rules across the openBIM lifecycle.



Speaker: Evgenii Ermolenko ⁽¹⁾
Supervisors: Bruno Figueiredo ⁽²⁾
Miguel Azenha ⁽³⁾

(1) University of Minho, EAAD, Lab2PT, ISISE, Guimarães, Portugal, 0009-0003-1228-9550

(2) Universidade do Minho, EAAD, Lab2PT, IN2PAST, Guimarães, Portugal, 0000-0001-8439-7065

(3) University of Minho, ISISE, ARISE, Department of Civil Engineering, Guimarães, Portugal, 0000-0003-1374-9427



Universidade do Minho
Escola de Engenharia

Universidade do Minho
Escola de Arquitetura, Arte e Design



Laboratório de Paisagens,
Património e Território



Fundação
para a Ciência
e a Tecnologia



DESIGN GRAMMARS SYSTEM SUPPORTING DECISION-MAKING IN CONSTRUCTION ACROSS PLATFORMS AND DISCIPLINES.

Roadmap

1. Why current BIM handovers fail to carry intent
2. What is Design Knowledge
 1. Design Rules
 2. Design State
3. The Knowledge Gap
4. Proposed “Design Grammars” system
 1. Goal
 2. Characteristics
 3. Knowledge Graph
 4. Ontology
 5. Framework
5. Prototype
 1. System Architecture
 2. Use Case. From client requirements to validated design state
6. Conclusion

How much design intent survives the next handover?

From Concept Design to Maintenance – stripping away design intent



LEGEND:



Retained
Design rule carried
forward



Partially retained
Design rule weakened or
not fully enforced



Lost
Design rule not carried
forward



Stripped
Design rule removed or
no longer tracked

DESIGN KNOWLEDGE

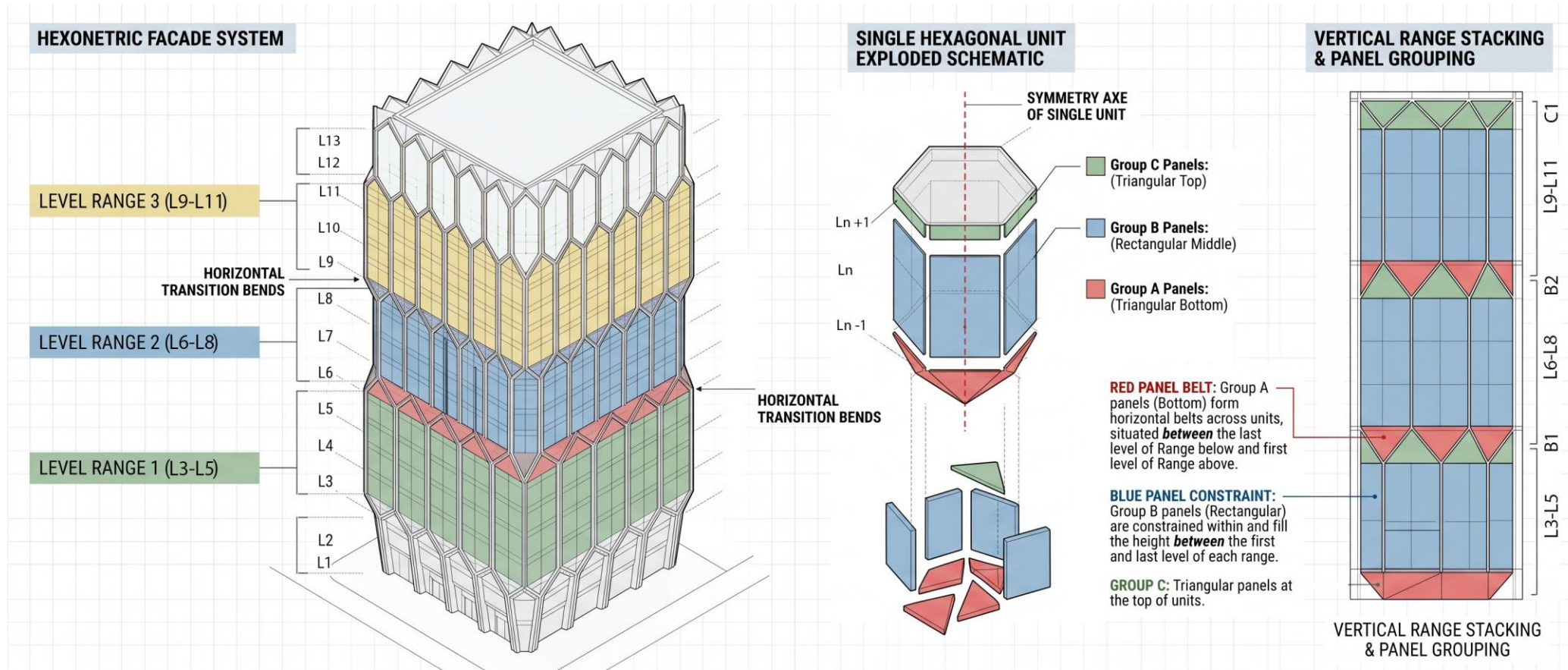
THE LOGIC BEHIND THE MODEL



Encoded machine-readable **Design Intent**, containing **Design Rules** (constraints, morphologic etc) that produce **Design States** (design solutions)

DESIGN RULES

Morphologic atoms of Design Knowledge



DESIGN RULES

Machine-readable SWRL expression

Class(?x) ^ Property(?x, ?y) ^ swrlb:greaterThan() -> **Property(?x, ?w)**

Rule Body

Rule Head

A – Facade System Generation

Rule A1 - Create pattern control points.

FacadeModule(?m) ^
hasUpperVertex(?m, ?v1) ^
hasLowerVertex(?m, ?v2)
-> PatternNode(?v1) ^
PatternNode(?v2)

Rule A2 – Create facade modules

C Panel Topology Classification

Rule C1 - Identify bottom triangular panels

GlazingPanel(?p) ^
hasTopologyType(?p,
"TriangularBottom")
-> GroupA(?p)

B Geometric and Parametric Constraints

Rule B1 - Vertex Alignment with Levels

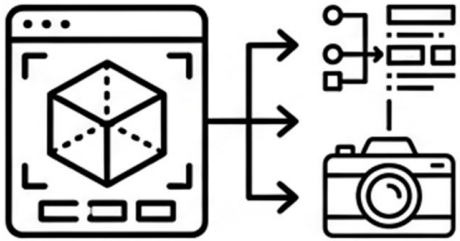
HexagonalUnit(?u) ^
hasVertex(?u, ?v) ^
liesOnLevelPlane(?v, ?l)
-> LevelAlignedVertex(?v)

Rule B2 - Unit Symmetry

D Performance and Envelope Validation Rules

Rule D1 – Panel Aspect Ratio Validation

GlassPanel(?p) ^
hasAspectRatio(?p, ?r) ^
swrlb:greaterThan(?r, 4.0)
-> ExcessivePanelSlenderness(?p)

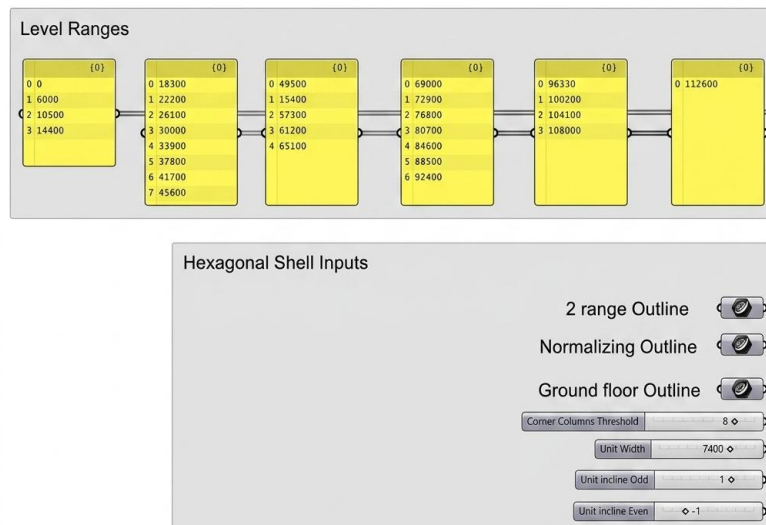


THE DESIGN STATE

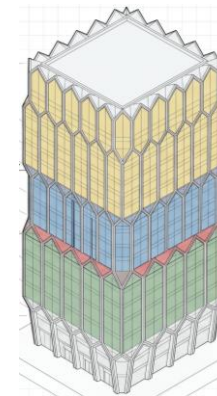
The Design State (Design Solution). Uniquely identified snapshot of a design's current parametric configuration — the set of input parameters (**defState**) and geometric objects (**objState**) captured from the parametric tool at a specific time.

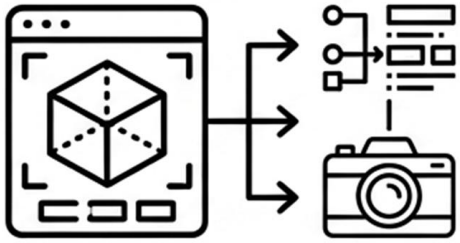


defState



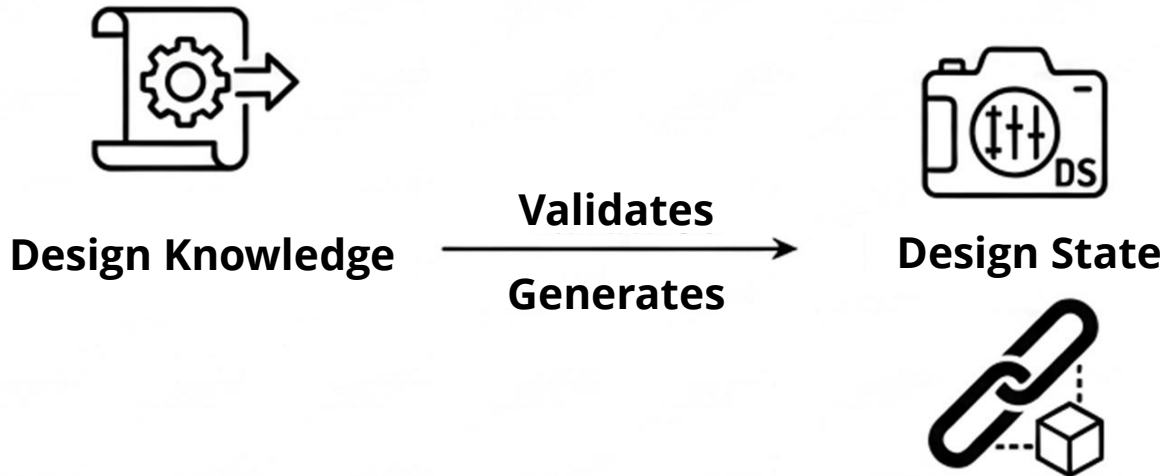
objState



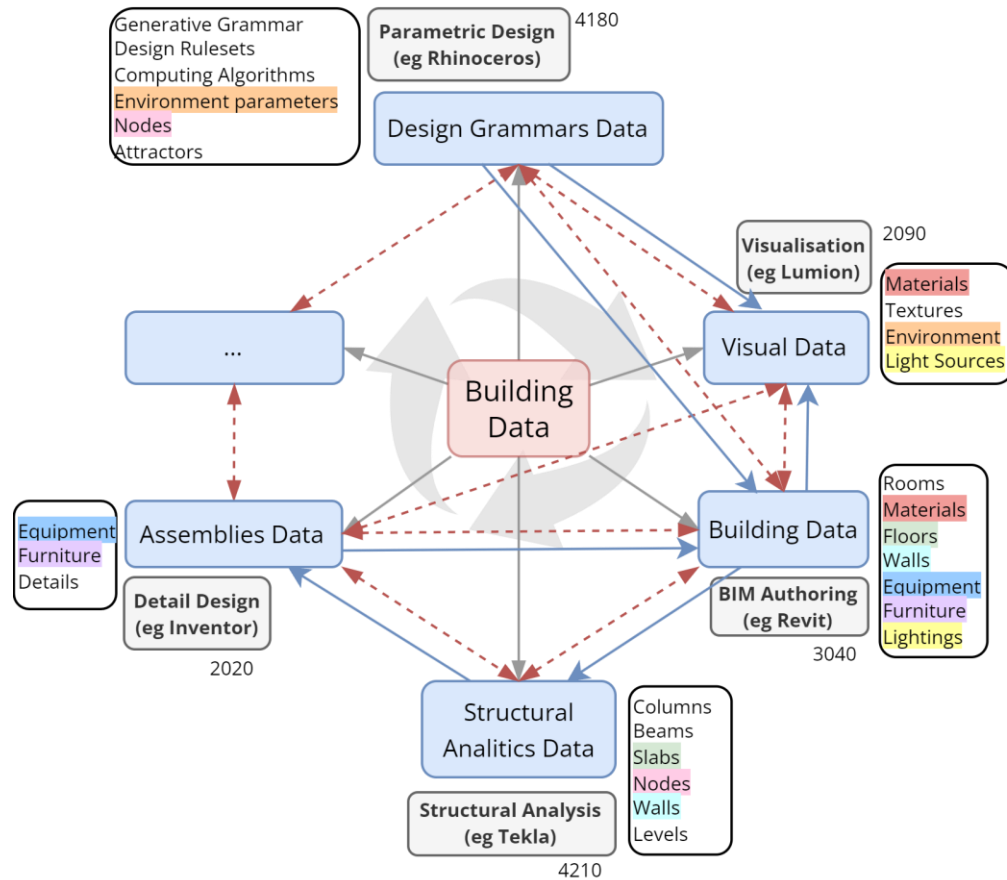


THE DESIGN STATE

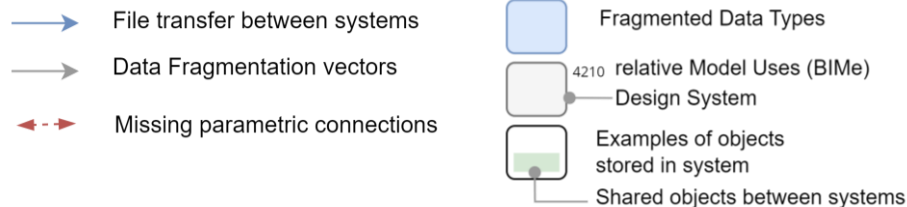
The Design State (Design Solution). Uniquely identified snapshot of a design's current parametric configuration — the set of input parameters (**defState**) and geometric objects (**objState**) captured from the parametric tool at a specific time.



Data fragmentation in Design Data Flow



Legend:



THE KNOWLEDGE GAP

- BIM manages information — but **design knowledge** stays **fragmented** across files, platforms and disciplines and **degraded** during design handover between design systems.
- **Luck of automated collecting and learning mechanisms** which would help to control design flow.

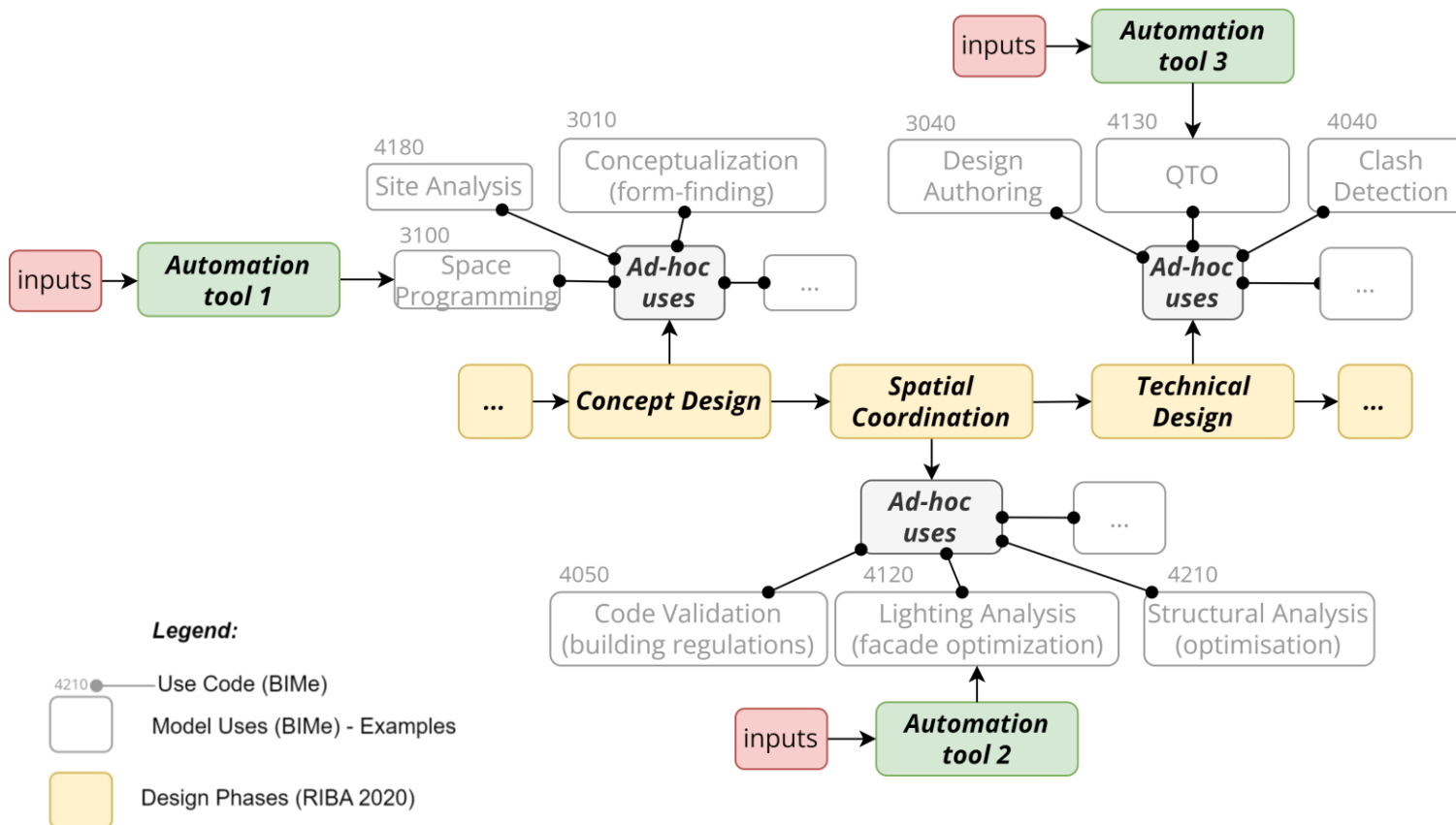
*Design Automation tools
limited to ad-hoc model uses
during Design process*

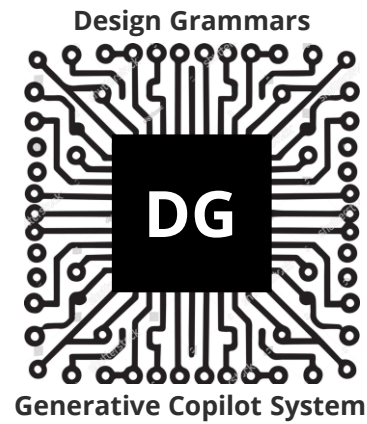
THE KNOWLEDGE GAP

Ad-Hoc Design Rules Application in BIM Automation

Parametric tools (Grasshopper / Dynamo) silo that logic as opaque, **non-interoperable** node graphs that die at export — **the rules are replicated per platform, never shared.**

* Model Uses comply with BIME Model Uses Table

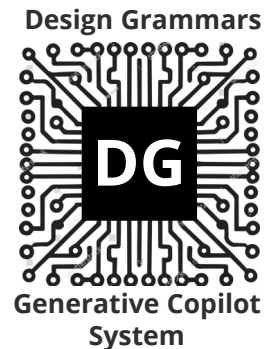




THE DESIGN GRAMMARS

PROPOSED DESIGN KNOWLEDGE MANAGEMENT SYSTEM

FROM COLLABORATIVE FILE-BASED EXCHANGE TO OBJECT-LEVEL KNOWLEDGE SHARING WITHIN INTEGRATED BIM

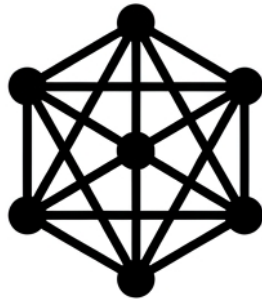


GOAL

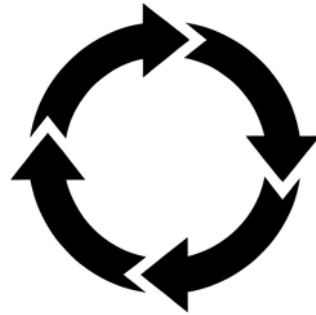
Build a Shared Knowledge Core that carries encoded design intent through a cross-platform environment of the BIM project and reuses it for validation and generation of design states during design development.

Example: "Hexagonal Units must be assigned per level ranges according to level boundaries and unit's vertices must coincide with level planes" travels as a reusable rule through architectural concept stage and BIM-model on Construction stage.

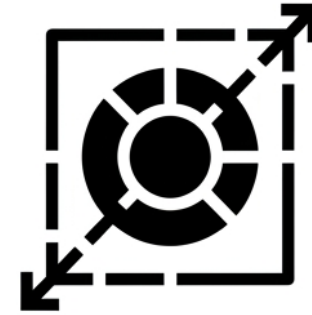
The Three Pillars of Design Grammars



**Explicit &
Machine-Readable**



**Generative &
Prescriptive**



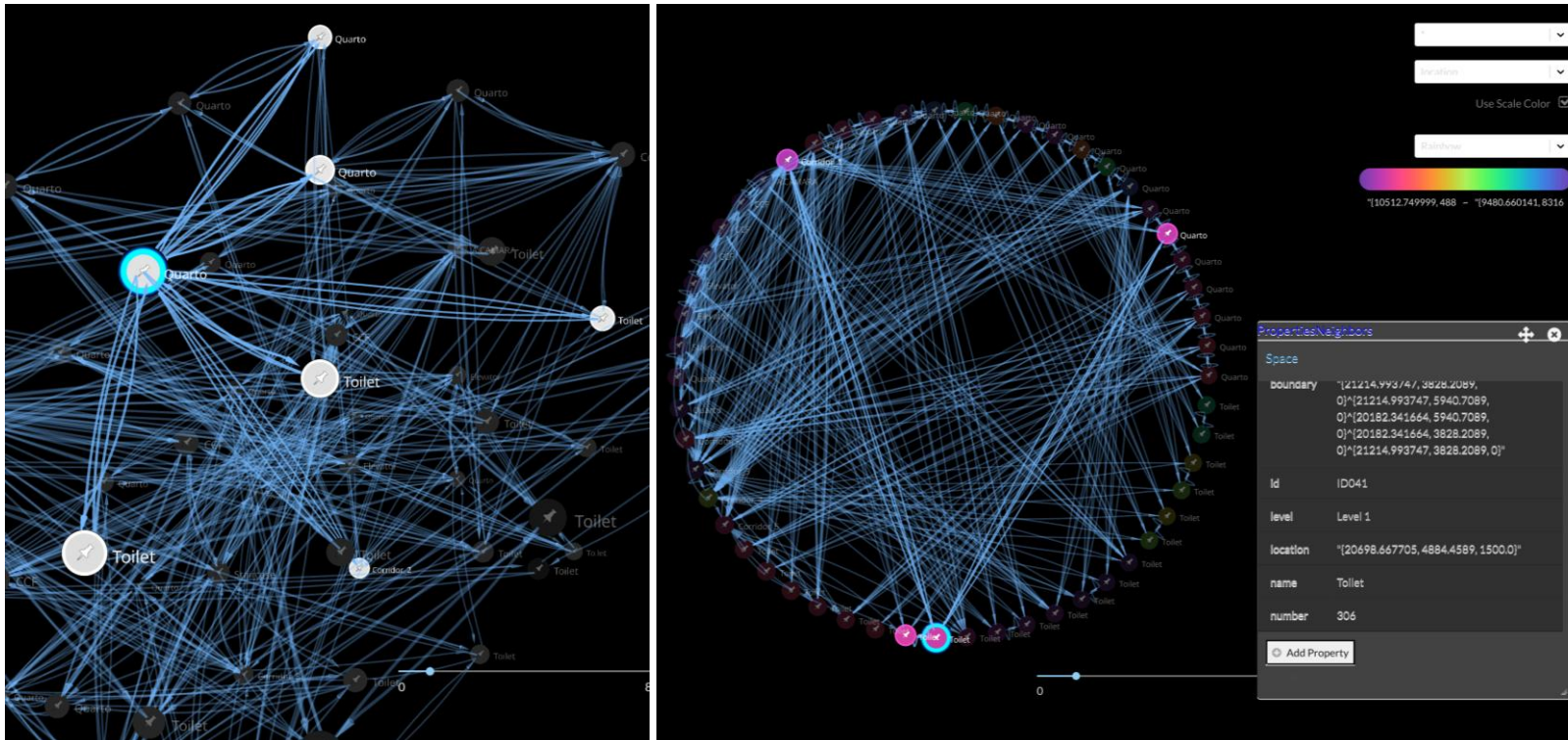
**Tool- & File-
Independent**

An Ontology-based Knowledge Graph

Knowledge Graph

A structured way to organize data

A **knowledge graph** is an organized representation of information that models real-world entities and relationships between them. It maps data visually as a network of connected labeled entities with properties as nodes and edges, so the data keeps its meaning and context.



Ontology

Organized domain knowledge

Ontology is a structured, machine-readable framework that organizes knowledge within a specific domain. It consists of:



Classes: The concepts or types of things (e.g., Employee, Manager).



Relationships (Object Properties): How these classes connect to one another (e.g., Manages, Works For).



Properties (Data Properties): Describing classes with specific value (literal)



Instances: The actual, concrete entities (e.g., the person "John Doe").

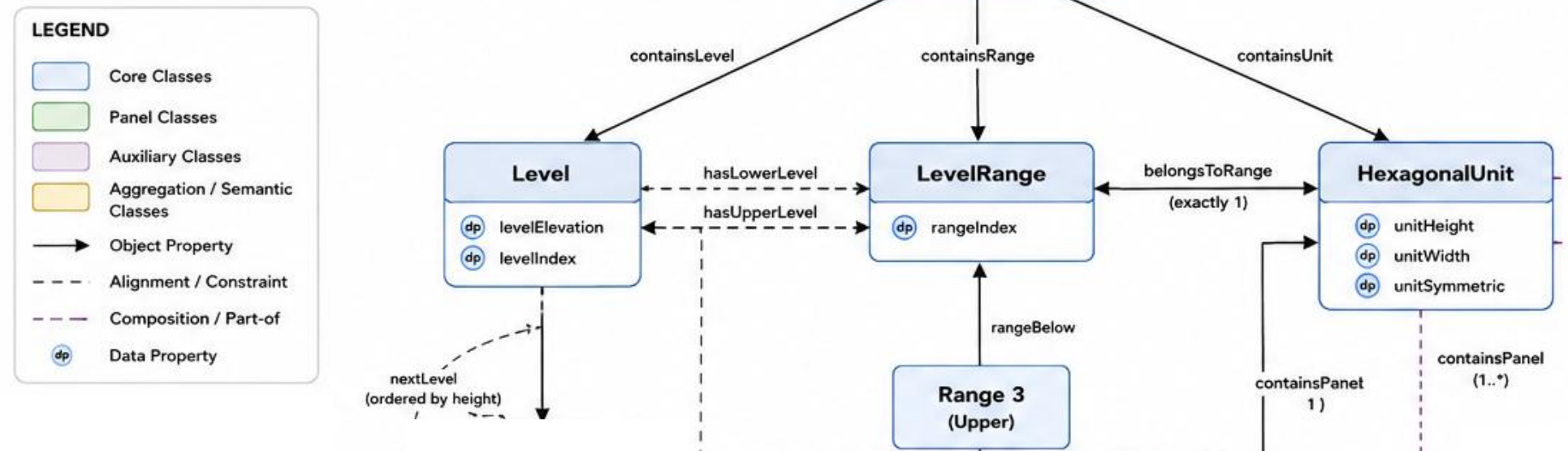
Ontology

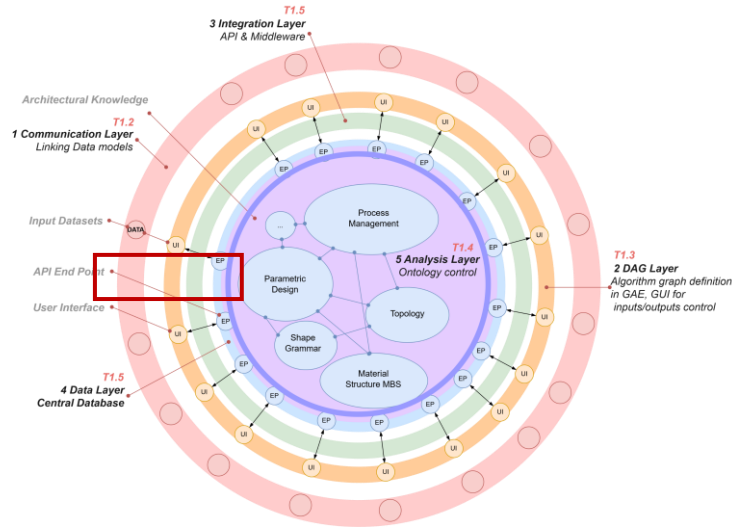
Organized domain knowledge

OntoGraph - Generated ontology from the encoded design intent

HEXAGONAL FACADE ONTOLOGY

Core Concepts, Relationships and Properties



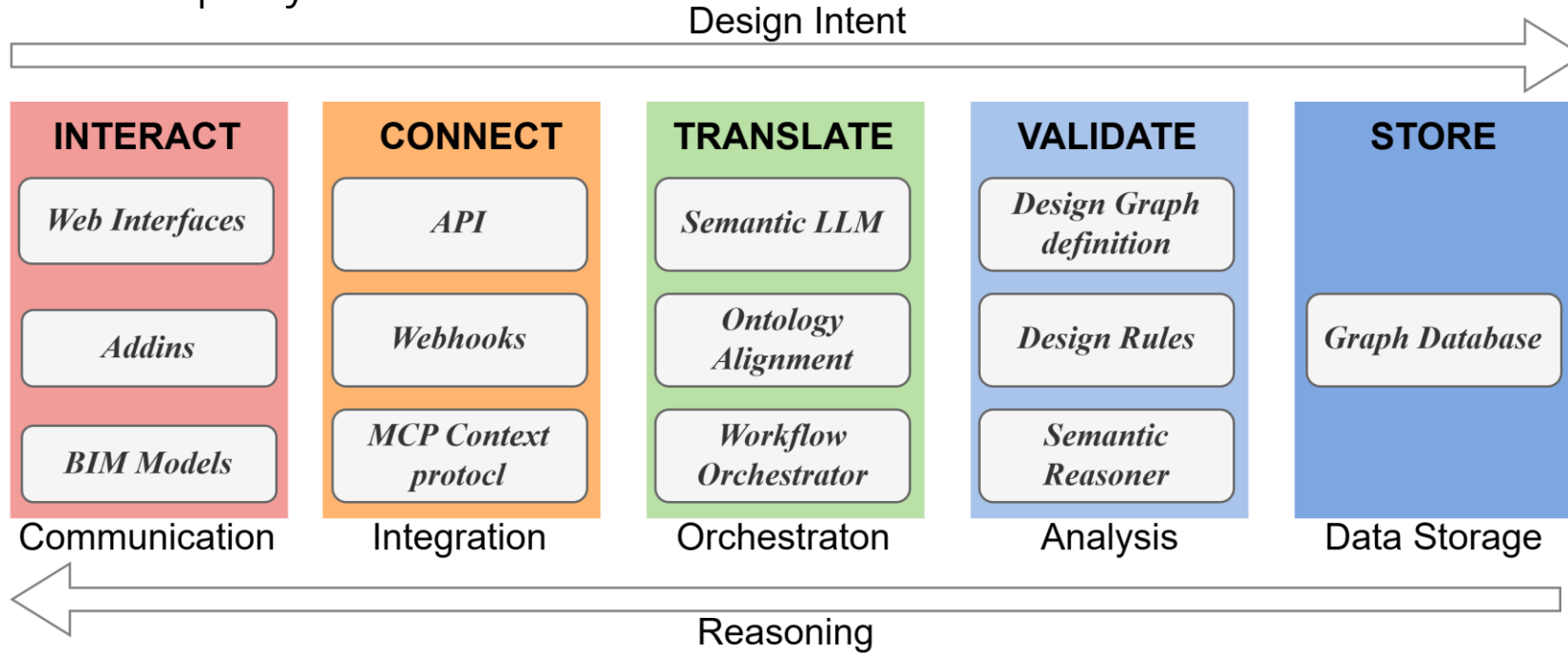


DG SYSTEM FRAMEWORK

5 LAYERS

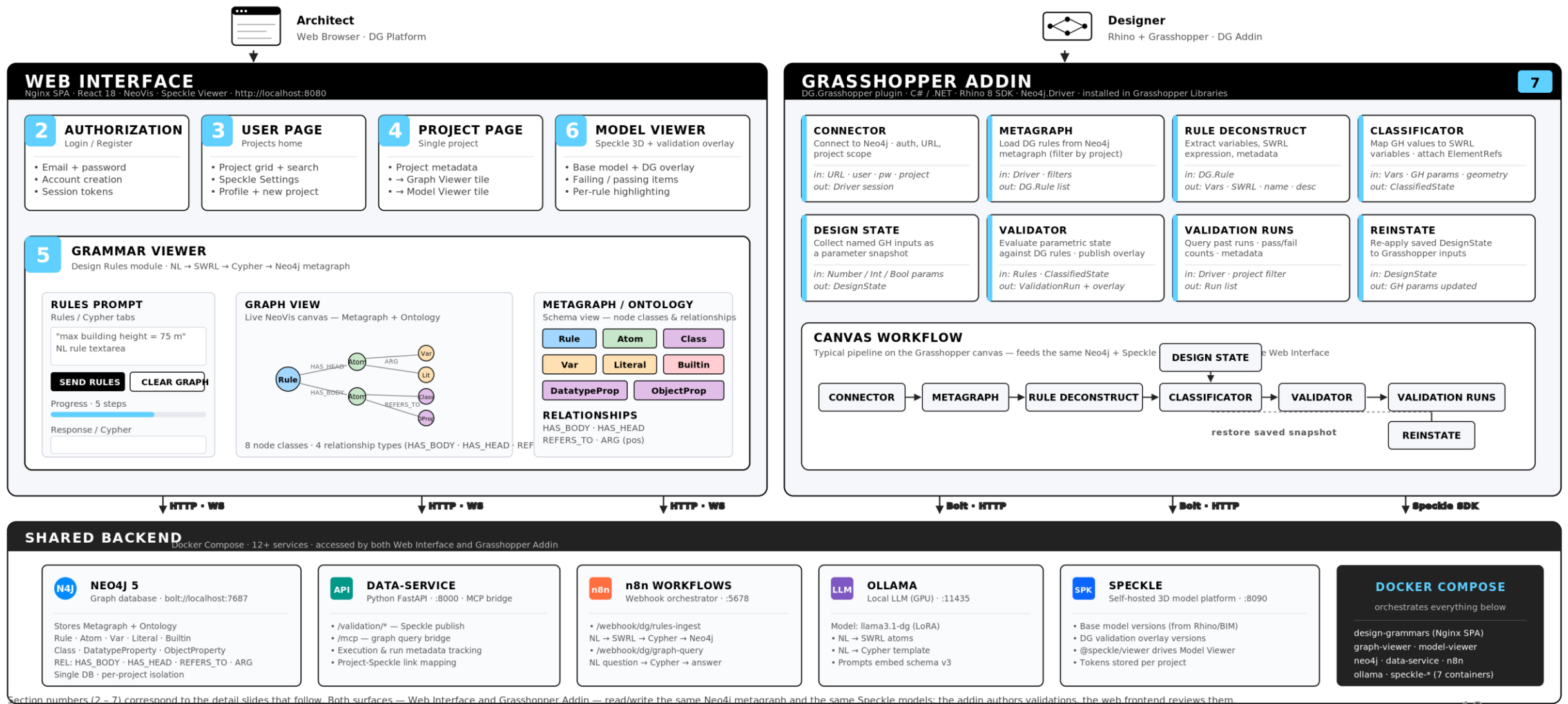
FROM SEMNTICS TO VALIDATION

Core-Periphery data model



DG SYSTEM PROTOTYPE

ARCHITECTURE



DG SYSTEM PROTOTYPE

USE CASE. FROM CLIENT REQUIREMENTS TO VALIDATED DESIGN STATE

USE CASE — FROM CLIENT REQUIREMENTS TO VALIDATED DESIGN

Three stakeholders · four acts · Knowledge Graph → MetaGraph → ValidationGraph

1 CLIENT BRIEF

Requirements & constraints uploaded to DG

UPLOAD DESIGN BRIEF

Submit requirements & constraints, addressed to the Lead Architect.

NL TEXT INPUT

FILE UPLOAD

→ Transposed into the Knowledge Graph as Requirement / Constraint entities

2 RULE AUTHORIZING

NL prompts → SWRL → Cypher → MetaGraph

SYNTHESIZE RULES

NL prompts in the Grammar Viewer → LLM crafts SWRL atoms + Cypher.

NL → SWRL

SWRL → CYPHER

LLM ASSIST

→ Populates the MetaGraph; each Rule node is linked back to its source Requirement / Constraint

3 DESIGN BOOTSTRAP

Parametric inputs + DG objects generated

CONNECT DESIGN PLATFORM

Place DG Addin components in Grasshopper; DG bootstraps the parametric scaffold.

CONNECTOR

METAGRAPHCLASSIFICATOR + DESIGN STATE

→ DG emits initial parametric inputs & DG.Objects ready for interactive validation

4 VALIDATE & COMMUNICATE

Design states tracked · solution returned

REVIEW SOLUTION

Receives validated design proposal with explicit links to original brief.

SOLUTION REPORT

3D OVERLAY

→ Feeds revised requirements back into the Knowledge Graph (loop closes)

VALIDATE & COMMUNICATE

Reviews validation runs in Model Viewer, approves the chosen design state and composes the response to the Client.

RUN AUDIT

RULE-LEVEL DIFF

→ Knowledge Graph updated with outcome

RECORD & ITERATE STATES

Each design variant captured by VALIDATOR; changes tracked in the ValidationGraph with full provenance.

VALIDATOR

VALIDATION RUNS

REINSTATE

→ Failing / passing items pushed to Speckle

CLIENT
Provides brief
Receives result

- Domain expert
- Owns goals & constraints
- Non-technical user

LEAD ARCHITECT
Authors design rules
Validates outcome

- Curates requirements
- Drives rule synthesis
- Approves design states

COMPUTATIONAL DESIGNER
Wires design platform

- Owns parametric model
- Drives DG Addin
- Iterates design states

SYSTEM STATE · graph evolution: single source of truth; each act mutates a different sub-graph

KNOWLEDGE GRAPH

CREATE
Requirements & Constraints transposed from Client NL + file input.



METAGRAPH (SWRL)

POPULATE
Rule · Atom · Var · Literal · Builtin nodes; Linked back to the Knowledge Graph.



DESIGN SCAFFOLD

EMIT
Parametric inputs & DG.Objects derived from MetaGraph variables.



VALIDATION GRAPH + KNOWLEDGE GRAPH

TRACK · APPROVE · UPDATE
DesignState · ValidationRun nodes; outcome flows back to Knowledge Graph.



→ approved RunResult writes DECISION back to the originating Requirement (loop closes)

DG SYSTEM PROTOTYPE

PHASE 1-2. Client Brief and Rule Authoring.

Web interface. Grammar Viewer

← Project TestA

Grammar Viewer

Mode

▼ Insert Rules

Graph View

MetaGraph

Rules Prompt

Cypher Expression

Enter Design Rules in NL text

Send Rules

Clear the Graph

Send rules

LLM parse

Write graph

Annotate properties

Reload graph

Response

Workflow response will appear here...

Workflow Cypher

Cypher used to build the graph will appear here...

```
graph TD
    true((true)) --> violatesMaxHeight((violatesMaxHeight))
    true --> R_BUILDING_MAX_HEIGHT_80_V_H1((R_BUILDING_MAX_HEIGHT_80_V_H1))
    R_BUILDING_MAX_HEIGHT_80_V_H1 --> R_BUILDING_MAX_HEIGHT_80_V_A2((R_BUILDING_MAX_HEIGHT_80_V_A2))
    R_BUILDING_MAX_HEIGHT_80_V_H1 --> R_BUILDING_MAX_HEIGHT_80_V((R_BUILDING_MAX_HEIGHT_80_V))
    R_BUILDING_MAX_HEIGHT_80_V_A2 --> hasHeightM((hasHeightM))
    R_BUILDING_MAX_HEIGHT_80_V --> R_BUILDING_MAX_HEIGHT_80_V_A1((R_BUILDING_MAX_HEIGHT_80_V_A1))
    R_BUILDING_MAX_HEIGHT_80_V --> R_BUILDING_MAX_HEIGHT_80_V_A3((R_BUILDING_MAX_HEIGHT_80_V_A3))
    R_BUILDING_MAX_HEIGHT_80_V_A1 --> Building((Building))
    R_BUILDING_MAX_HEIGHT_80_V_A3 --> greaterThan((greaterThan))
    Building --> violatesMinWindowCount((violatesMinWindowCount))
    violatesMinWindowCount --> R_LIVINGUNIT_MIN_WINDOW_2_V_H1((R_LIVINGUNIT_MIN_WINDOW_2_V_H1))
    R_LIVINGUNIT_MIN_WINDOW_2_V_H1 --> R_LIVINGUNIT_MIN_WINDOW_2_V((R_LIVINGUNIT_MIN_WINDOW_2_V))
    R_LIVINGUNIT_MIN_WINDOW_2_V --> R_LIVINGUNIT_MIN_WINDOW_2_V_A1((R_LIVINGUNIT_MIN_WINDOW_2_V_A1))
    R_LIVINGUNIT_MIN_WINDOW_2_V --> R_LIVINGUNIT_MIN_WINDOW_2_V_A2((R_LIVINGUNIT_MIN_WINDOW_2_V_A2))
    R_LIVINGUNIT_MIN_WINDOW_2_V --> R_LIVINGUNIT_MIN_WINDOW_2_V_A3((R_LIVINGUNIT_MIN_WINDOW_2_V_A3))
    R_LIVINGUNIT_MIN_WINDOW_2_V_A1 --> LivingUnit((LivingUnit))
    R_LIVINGUNIT_MIN_WINDOW_2_V_A2 --> hasWindowCount((hasWindowCount))
    R_LIVINGUNIT_MIN_WINDOW_2_V_A3 --> lessThan((lessThan))
    hasWindowCount --> 2((2))
    lessThan --> 2
```

Node details

Rule

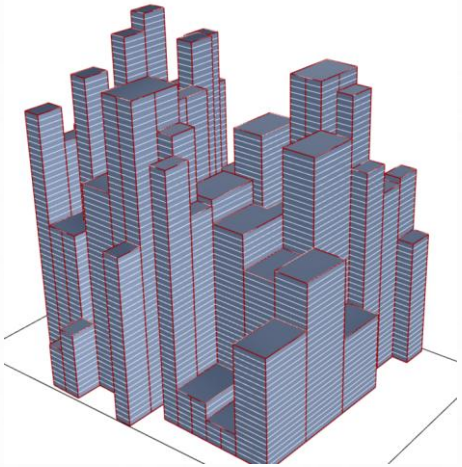
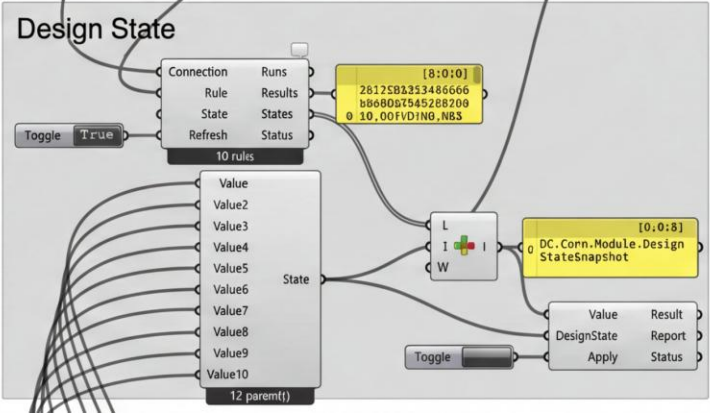
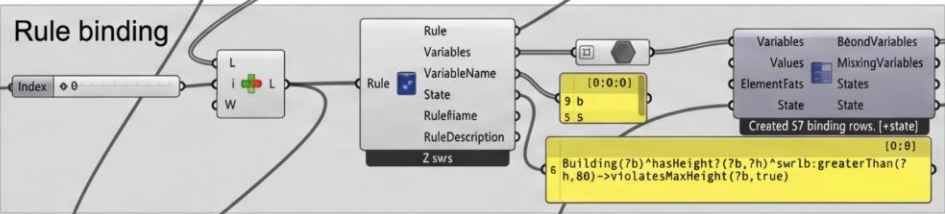
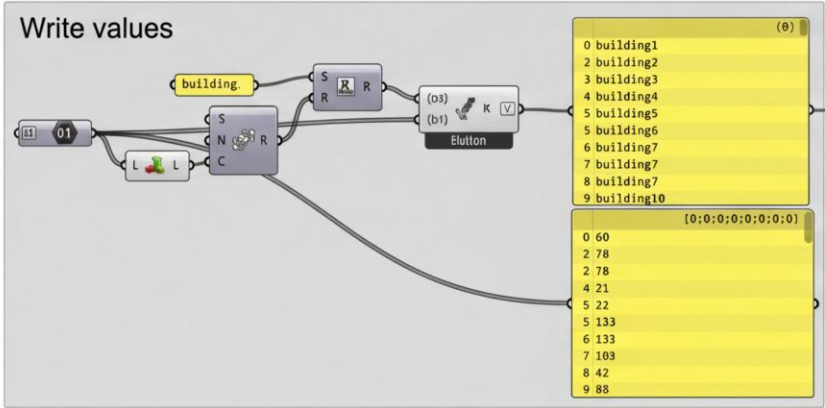
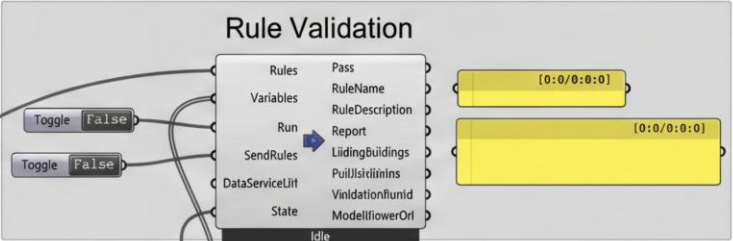
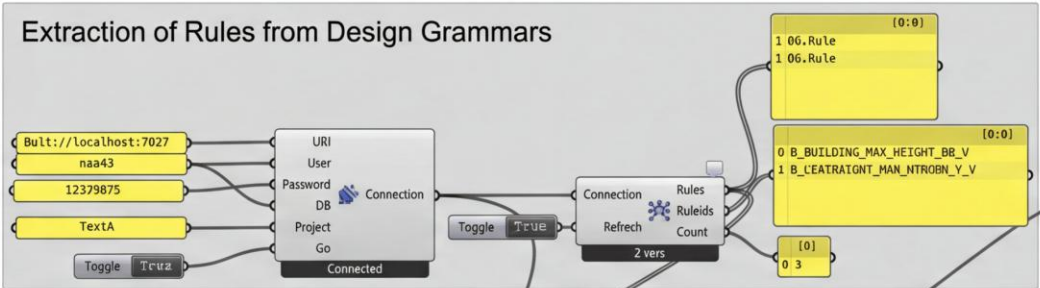
Key	Value
<id>	88
Rule_Id	"R_LIVINGUNIT_MIN_WINDOW_2_V"
description	"All living units must have at least 2 windows"
graph	"Metagraph"
kind	"violation"
project	"TestA"
text	"LivingUnit(?u)^hasWindowCount(?u,?w)^swrlb:lessThan(?w,2)->violatesMinWindowCount(?u,true)"

Add property

DG SYSTEM PROTOTYPE

PHASE 3. Design Bootstrap.

Grasshopper Addin



DG SYSTEM PROTOTYPE

PHASE 4. Validation and Communication.

Web Interface. Design State Viewer

← Project

Speckle connector >

Model Viewer

TestA

VALIDATION RUN

Run Id

43f8c162d21745e2abfa4d7f7524fa63

Date Created

06.05.2026, 09:14:56

Base Model

4c546c0771

Validation Model

a6d1e0c5da

Failing

31

Passing

26

> Failing items (31)

> Passing items (26)

RULE

Rule_Id

R_BUILDING_MAX_HEIGHT_80_V

Description

Max height must be 80 meters

✓ Failing

✓ Passing

□ Base model

Fail

Pass

Base

Fail α

80%

Pass α

60%

Base α

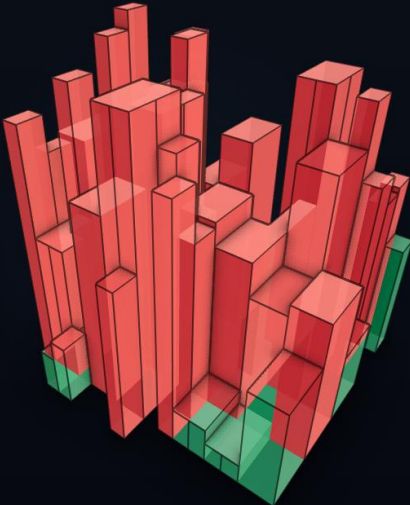
30%

Q

Select by Id...

Isolate

Hide



VALIDATION RUNS 20

GROUP BY Rule Design State

✓ R_BUILDING_MAX_HEIGHT_80_V 18

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

R_BUILDING_MAX_H... ✕

10.05.2026, 17:51:26

09.05.2026, 13:43:22

06.05.2026, 14:07:36

06.05.2026, 09:14:56

06.05.2026, 09:13:24

06.05.2026, 08:53:51

05.05.2026, 23:12:10

05.05.2026, 18:54:57

05.05.2026, 18:52:42

Constraint

Constraint

Constraint

Constraint

Constraint

Constraint

Constraint

Constraint

Constraint

✓ R_LIVINGUNIT_MIN_WINDOW_2_V 2

R_LIVINGUNIT_MIN_... ✕

R_LIVINGUNIT_MIN_... ✕

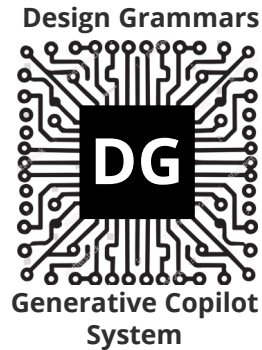


22

CONCLUSION

Contribution

A working prototype that encodes, manages and reuses grammar knowledge across the design lifecycle.

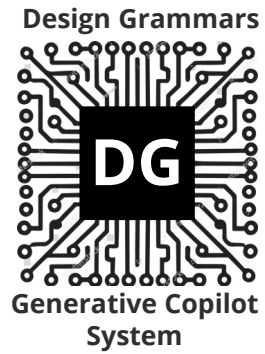


What works

- Multimodal Rule ingestion
- Structured ingestion of project documentation
- Iterative validation of design states against rule sets
- Review validation results
- Design states record and reinstating parametric model

Limitations & next steps

- SWRL coverage limited to ClassAtom / DataPropertyAtom / BuiltinAtom — next: extend to full SWRL body coverage
- Academic evaluation only — production stress-testing pending



**THANK YOU
FOR YOUR
ATTENTION**